Note on Data Encryption

Riccardo Lo Iacono

October 27, 2026

Contents.

1 Introduction to cryptography and codes **1**
1.1 Some prerequisites 1

2 Public key cryptography **5**
2.1 RSA 5
2.2 Diffie-Hellman 6
2.3 ElGamal 6
2.4 The hard problems underlying public key encryption . . 7

3 Notion of projective geometry **9**
3.1 The projective space 9
3.2 ElGamal on elliptic curves 12

4 Digital signature **13**
4.1 Using RSA to sign a document 13
4.2 Using ElGamal to sign a document 13

5 Factorization algorithms **15**
5.1 Pollard factorization algorithms 15

References **17**

– 1 – Introduction to cryptography and codes.

The notion of cryptography is an old one; it derived from the necessity of hiding the content of a message when communicating through a medium (a channel for instance). In other words, cryptography was born as a way to make communications more secure. Similar to cryptography, stenography is another way to make communications secure, but the procedure behind it are completly different.

These notes will cover exclusively concepts of cryptography, and are meant to provide a broad view of the field. In addition, some code theory will be discussed half-way through these notes.

– 1.1 – Some prerequisites.

Although these notes are meant to give a broad view to the field, some mathematical background is somewhat required. For this reason, the present section will discuss some basic concepts of abstract algebra, as well as some concepts of discrete math.

Here, we introduce the notions of group, abelian group, ring and field; later, section 3, we discuss some concepts of projective geometry since this provide the basic tools for modern cryptography.

– 1.1.1 – Groups, rings and fields.

To give the most accessible idea of what each of groups, rings and fields are, we can think of this as sets on which some operations are defined; in addition, such operations must satisfy some properties.

Formally speaking, given a set G and a binary operation $* : G \to G$[1], we say that $(G, *)$ forms a group if all of the following hold.

1. There exists an element $\mathbf{1}_G$ such that $a * \mathbf{1}_G = \mathbf{1}_G * a = a, \forall a \in G$. In other words, G must have a neutral element.

2. For all $a \in G$ there must exists a^{-1} such that $a * a^{-1} = a^{-1} * a = \mathbf{1}_G$. Meaning that each element must have an inverse.

3. The operation $*$ must be associative. That is, for all $a, b, c \in G$ it holds $a * (b * c) = (a * b) * c$.

Additionally, if $*$ is commutative, i.e., $a * b = b * a$, we say that $(G, *)$ is an abelian (or commutative) group.

[1] Note that the use of $*$ for the binary operation should not fool the reader in to thinking it as the well known multiplication. Instead, it is a placeholder for any well defined operation.

As for sets, groups may contain some smaller groups, to which we refer to as sub-groups. Formally, given $(G, *)$ a group, we say that $H \subseteq G$ forms a group under $*$ if $\mathbf{1}_G \in H$ and $(H, *)$ forms a group.

Beyond the set-theoretic relation, groups and subgroups are connected by a much more foundamental relation, that regarding their size. This result, known in the literature as *Lagrange's theorem for groups*, states that a finite group G, i.e. a group with finitely many elements, can only have subgroups H such that the number of elements in H divides the number of elements in G. We Formally state Lagrange's theorem below.

Theorem (Lagrange's theorem for groups): *Let $(G, *)$ be a group, with $|G| < \infty$. Then, any subgroups $(H, *)$ of $(G, *)$ is such that*

$$|H||G|.$$

Although groups have some nice properties as the whole group-theory shows, more often than not we need something more powerful. To provide for this need, some more advanced structures were developed, the simplest of which is known as *ring*.

To undestand rings, we can see them as an extention of groups by an additional operation. More precisely, we say that $R, +, *$ is a ring if

1. $(R, +)$ form an abelian group.
2. $*$ is associative. That is, for all $a, b, c \in R$ it holds $a * (b * c) = (a * b) * c$.
3. $*$ distributes over $+$. In other words, if for all $a, b, c \in R$ we have that $a * (b + c) = a * b + a * c$.

Rings can sometimes satisfy some additional properties, depending on which they take different names. These additional properties are the following.

1. There exists a neutral element for $*$.
2. $*$ is commutative.
3. Each element in R as inverse with respect to $*$.

A ring with all these additional properties is called *field*. In these notes we denote a generic field by either $\mathbb{F}$ or $\mathcal{K}$, while finite fields will be denoted by $\mathrm{GF}(p)$, where p is usually a prime.

– 1.1.2 – Modular arithmetics.

Most of the groups and ring will be using in these notes, rise naturally from the notion of congruence modulo n, where n is an integer. These groups and rings will be denoted by $\mathbb{Z}_n$, or by $\mathbb{Z}_p$ if $n = p$ is prime.

But what is congruence? Put in simple words, given two integers a, b, we say that a is congruent b modulo n, if we can write a as the sum of b and some multiple of n. That is,

$$a \equiv b \pmod{n} \iff \exists k \in \mathbb{Z} : a = b + kn.$$

Using the above definition, we get

$$\mathbb{Z}_n = \{m \in \mathbb{N} \mid m < n\}.$$

For instance, $\mathbb{Z}_{11} = \{0, 1, \ldots, 9, 10\}$. Sometimes when dealing with rings we will write $\mathbb{Z}_n^*$ to mean the group/ring $\mathbb{Z}_n \setminus \{0\}$.

In most of the algorithms we will discuss, we often make use of the gcd between two given integers. Here we first recall what GCD is, and provide two algorithm to compute it.

Let a, b be two integers. We say that an integer d is the GCD of a, b if the following hold:

$$d|a \text{ and } d|b \tag{1}$$

and for any other integer d' such that Equation (1) holds, we have

$$d'|d.$$

In other words, the d is the GCD of a, b if any integer that divides a, b also divides d.

But how do we compute the GCD of two integers? A result in abstract algebra tells us that we can compute d as the difference between some multiple of a and some multiple of b. This result is known as *Bézout identity*. More precisely Bézout identity states that there exists two integers δ and ϵ, such that $d = \gcd(a, b) = a\delta + b\epsilon$. δ and ϵ are called Bézout coefficients.

In practise we compute the GCD by means of the Euclidean algorithm. Two variants exists, a simple one which returns just the GCD, and one which returns both the GCD and the Bézout coefficients. Here we present both, although for our needs the latter is preferred.

Algorithm 1 Euclidean algorithm

Input: Two integers a and b, with $a > b$.
Output: The greatest GCD of a and b.
Compute the sequence $\{r_i\}_{i \in \mathbb{N}}$ as follow: $r_0 \leftarrow a, r_1 \leftarrow b$
  while $r_k \geq 1$ **do**
    $r_{k+1} \leftarrow r_{k-1} - q_i r_k$
  end
return the last computed r_k

Algorithm 2 Euclidean algorithm

Input: Two integers a and b, with $a > b$.
Output: The greatest GCD of a and b, and the Bézout coefficients.
Compute the sequences $\{r_i\}_{i\in\mathbb{N}}, \{r_i\}_{i\in\mathbb{N}}, \{t_i\}_{i\in\mathbb{N}}$ as follow:

$r_0 \leftarrow a, r_1 \leftarrow b$
$s_0 \leftarrow 1, s_1 \leftarrow 0$
$t_0 \leftarrow 0, t_1 \leftarrow 1$
while $r_k \geq 1$ **do**
$r_{k+1} \leftarrow r_{k-1} - q_i r_k$
$s_{k+1} \leftarrow s_{k-1} - q_i s_k$
$t_{k+1} \leftarrow t_{k-1} - q_i t_k$
end

return the last computed r_k as GCD and s_k, t_k as Bézout coefficients.

– 2 – Public key cryptography.

Until the first years of the '70s cryptography was mainly using a symmetric approach; that is, sender and receiver had the same encryption/decryption key to cypher their messages. But how should the key be choosed in the first place? This and other reasons, thanks to the contribution of Diffie and Hellman, this symmetric schema was soon replaced by what is known as *public key* cryptography.

Let us denote by $\mathcal{A}$ and $\mathcal{B}$ the two entities involved in the communication, and let $\mathcal{E}$ be a malicious entity trying to intercept the communication. Any public key schema follows the same principle:

1. $\mathcal{B}$ chooses a function f such that, apart for $\mathcal{B}$, its inverse is hard to compute.
2. Let m be the message $\mathcal{A}$ wants to send to $\mathcal{B}$; $\mathcal{A}$ computes $c = f(m)$ and transmits c to $\mathcal{B}$.
3. Upon receiving c, $\mathcal{B}$ who knows how to compute f^{-1}, computes $m = f^{-1}(c) = f^{-1}(f(m))$.
4. Even if $\mathcal{E}$ is able to get to c, it can't retrive m since f^{-1} is hard to compute.

The remainder of this section will focus on two of the most commonly used public key scemas, namely RSA and Diffie-Hellman. In Section 3, we describe how RSA and Diffie-Hellman can be implemented using elliptic curves.

– 2.1 – RSA.

Taking the name from the three researchers who developed it, Rivest, Shamir, and Adleman, RSA is one of the simplest, yet solid[2]algorithm in cryptography.

The idea behind RSA is the following: we choose two prime numbers p and q of similar size, i.e., their binary representation uses approximately the same numebr of bits, we compute their product n and use this value to cypher our message. In Procedure RSA, we give a more detailed description of the RSA algorithm.

What we are left with is showing that Line 3 of Procedure RSA, does in fact return m. First observe that

$$c^d = (m^e) = m^{ed}.$$

Using Bézout identity, we get

$$m^{ed} = m^{1-\mu\lambda} = m * m^{-\mu\lambda} = m * (m^{\lambda})^{-\mu}.$$

[2]Here by "solid" we refer to the fact that cracking RSA is somewhat unfeasable.

Procedure RSA

1. $\mathcal{B}$ chooses two prime numbers p and q with similar size, and computes $n = p * q$ and $\lambda = \text{lcm}(p-1, q-1)$. Additionally $\mathcal{B}$ chooses ϵ such that $\gcd(\lambda, \epsilon) = 1$. The pair (n, ϵ) represents $\mathcal{B}$'s public key.
2. $\mathcal{A}$ cyphers its message m, which is assumed to be an integer, by computing $c = m^{\epsilon} \pmod n$, and sends it to $\mathcal{B}$.
3. To retrieve m, $\mathcal{B}$ computes $m = c^d \pmod n$, where d is the Bézout coeffient of ϵ in the identity $1 = d\epsilon + \mu\lambda$.
4. If $\mathcal{E}$ gets to c, although has access to $\mathcal{B}$'s public key, can't possibly retrieve m. The only possibility is for $\mathcal{E}$ to factor n, which is a well known hard problem.

Recall that by definition $\lambda = \text{lcm}(p-1, q-1)$, hence there exists r, s such that $\lambda = (p-1)r = (q-1)s$. By using Fermat's theorem, which roughuly states that $x^p \equiv x \pmod p$, we get

$$m^{\lambda} = m^{(p-1)r} = (m^{(p-1)})^r = 1^r = 1 \pmod p.$$

By the same logic, we get that $m^{\lambda} = 1 \pmod q$. Therefore,

$$c^d = m^{ed} = (m^{\lambda})^{-\mu} = m * 1 \pmod p = m * 1 \pmod q = m.$$

– 2.2 – Diffie-Hellman.

Born by the collaboration between Diffie and Hellman, the Diffie-Hellman key exchange protocol was the first public key exchange algorithm. Roughuly speaking the algorithm is the following: we select a prime number p and consider and choose an element $\gamma \in \text{GF}_p$, both parties, in our case $\mathcal{A}$ and $\mathcal{B}$, choose a private key, let these be a for $\mathcal{A}$ and b for $\mathcal{B}$, and compute the public key by taking the a-th (b-th resp.) power of γ.

A more detailed description of the algorithm is shown in Procedure Diffie-Hellman-Key-Exchange.

As stated, Diffie-Hellman is more a public key exchange protocol rather then an encryption schema; though, as described in the next section, when combined with Vernam cyphering algorithm, it allows to build a solid public key encryption schema known as the *ElGamal* encryption schema.

– 2.3 – ElGamal.

As anticipated in Section 2.2, the ElGamal encryption schema is based on the Diffie-Hellman algorithm to echange the key, what is left is undestanding how the message is encrypted/decrypted.

Procedure Diffie-Hellman-Key-Exchange

1 Let $\mathcal{T}$ be a trusted third party. $\mathcal{T}$ chooses a prime number p and chooses an element $\gamma \in \mathrm{GF}_p$.
2 Consider $\mathcal{B}$; it chooses an integer b and computes $\beta \equiv \gamma^b \pmod{p}$, which would represent its public key. In the same manner $\mathcal{A}$ computes its public key $\alpha \equiv \gamma^a \pmod{p}$.
3 Both $\mathcal{A}$ and $\mathcal{B}$ can compute the same private key $\kappa_{\mathcal{A}\mathcal{B}} \equiv \beta^a \pmod{p}$, since $\beta^a = \alpha^b$.
4 Assuming $\mathcal{E}$ intercepts either of α and β, it won't be able to retrive $\kappa_{\mathcal{A}\mathcal{B}}$ since a and b are unknown.

Using the same notation as before, the sender, which we assume to be $\mathcal{A}$, computes $c = \kappa_{AB} * m$ and sends it to $\mathcal{B}$, where m is assumed to be an integer. On the recieving side, $\mathcal{B}$ retrives m by computing $m = \kappa_{AB}^{-1} * c$. The question is, how is κ_{AB}^{-1} obtained? To answer this question observe that $\gcd(\kappa_{AB}, p) = 1$, meaning that we can use Bézout's identity to find x and y such that $1 = \kappa_{AB}x + py$.

In Procedure ElGamal we give the complete algorithm.

Procedure ElGamal

1 By using Procedure Diffie-Hellman-Key-Exchange, let $\mathcal{A}$ and $\mathcal{B}$ compute their public keys α and β, and the secrete key κ_{AB}.
2 Let m be an integer that represents the message $\mathcal{A}$ wants to send to $\mathcal{B}$. $\mathcal{A}$ computes $c = \kappa_{AB} * m$ and sends it to $\mathcal{B}$.
3 Recieving $c, \mathcal{B}$ first computes κ_{AB}^{-1} solving Bézout's identity, then retrives $m = \kappa_{AB}^{-1} * c$.
`/* On the attacker side...  */`
4 Even if $\mathcal{E}$ obtains c, since it has no access to κ_{AB}, retriving m is impossible.

– 2.4 – The hard problems underlying public key encryption.

Now that we have seen a couple of public key encryption algorithm, a question should pop inside the reader's head: what garantees do we have about these algorithms?

It should come to no surprise that the answer is given by math. Precisely, by the hardness of the problems underlying these algorithms. For instance, the security of RSA is given by the hard problem of integers factorization,

which asks us, given an integer n, to find its factorization into primes. The Diffie-Hellman algorithm, and therefore ElGamal, are based on the *discrete algorithm* problem, which roughuly asks, given and integer α, to find a such that $\alpha \equiv \gamma^a \pmod{p}$, where γ and p are known.

These we presented are just a couple of several public key encryption algorithm. The interested reader can easely find a whole literature out there.

– 3 – Notion of projective geometry.

As anticipated in the introduction, modern days cryptography is based on the notion of elliptic curve. Before we can define what these curves are, we have to understand what a projective space is.

– 3.1 – The projective space.

Projective spaces are deeply rooted in art, as these were implicitly used when dealing with perspective. As it turns out, perspective is nothing else but a consequence of how the projective space works.

Formally speaking, an n-dimensional projective space over a field $\mathcal{K}$ is defined as the set of non-empty *rays* $\mathbf{v} \in \mathcal{K}^{n+1}$. More precisely

$$\mathbb{P}^n(\mathcal{K}) = \{P = \langle \mathbf{v} \rangle \mid \mathbf{0} \neq \mathbf{v} \in \mathcal{K}^{n+1}\},$$

where $\mathbf{v} = [x_0, \ldots, x_n], x_i \in \mathcal{K}$.

Points in $\mathbb{P}^n(\mathcal{K})$ are represented using two types of coordinates:

1. *Homogeneous* coordinates, by which a point Q differs from P if and only if $Q \neq \lambda P$. In other words, using homogeneous coordinates, all points that differ by a multiplicative constant represent the same point.

2. *Non-homogeneous* coordinates, by which P is written as

$$P = \begin{cases} [1, {}^{x_1}\!/_{x_0}, \ldots, {}^{x_n}\!/_{x_0}], \text{ if } x_0 \neq 0; \\ [0, x_1, \ldots, x_n], \text{ if } x_0 = 0. \end{cases}$$

When using non-homogeneous coordinates, we say that $P = [x_0, \ldots, x_n]$ is a proper point, or equivalently, an affine point if $x_0 \neq 0$; we say that P is an improper point, or equivalently, a point at infinity if $x_0 = 0$.

Our discussion on elliptic curves will involve points in $\mathbb{P}^2(\mathcal{K})$, often represented by non-homogeneous coordinates. For this reason, to ease the notation a bit, we write $P = [1, {}^{x_1}\!/_{x_0}, {}^{x_2}\!/_{x_0}]$ as $P = [1, x, y]$, where $x = {}^{x_1}\!/_{x_0}$ and $y = {}^{x_2}\!/_{x_0}$. For point at infinity we write P as $P = [0, X, Y]$.

– 3.1.1 – Homogeneous polynomials.

Evaluating polynomial in a projective space is tricky, depending on the polynomial at hand. Consider a polynomial in which its monomyals have different degrees, take for instance the following:

$$f(X) = f(x_0, x_1, x_2) = x_0 - x_2 x_1.$$

We would expect that, evaluating it a the points $P = [2, -1, 1]$ and $Q = 3P = [6, -3, 3]$, which are equal under the homogeneous interpretation, we would get the same result. Instead we get

$$f(P) = 2 - (-1 * 1) = 3 \text{ and } f(Q) = 6 - (-3 * 3) = 15.$$

As the reader might have understood, the issue is that polynomials like the one above are not homogeneous. That is, they don's satisfy the following equation:

$$f(\lambda x_0, \lambda x_1, \lambda x_2) = \lambda^d f(x_0, x_1, x_2)$$

where d is the degree of the monomyals.

Let $f(X)$ and $g(X)$ be two homogeneous polynomials, and let d be the degree of their monomyals. Let us see what happens when we compute $F(X) = {}^{f(X)}/_{g(X)}$; we get

$$F(\lambda X) = \frac{f(\lambda X)}{g(\lambda X)} = \frac{f(\lambda x_0, \lambda x_1, \lambda x_2)}{g(\lambda x_0, \lambda x_1, \lambda x_2)} = \frac{\lambda^d f(x_0, x_1, x_2)}{\lambda^d g(x_0, x_1, x_2)} = F(X).$$

That is, $F(X)$ doesn't care about multiplicative constants.

Luckly for us, in the projective space any polynomial can be seen as the ratio of two homogeneous polynomials. For instance, the following non-homogeneous polynomial

$$F(x, y) = 3x^2 - 2y + 1$$

can be transformed into an homogeneous polynomial as follow: we replace x by ${}^{x_1}/_{x_0}$ and y by ${}^{x_2}/_{x_0}$, compute the lowest common multiple and obtain

$$F(x, y) = 3x^2 - 2y + 1 = \frac{3x_1^2 - 2x_0x_2 + x_0^2}{x_0^2} = F(x_0, x_1, x_2).$$

Evaluating $F(x_0, x_1, x_2)$ at P and Q, we get the same result of ${}^3/_4$.

– 3.1.2 – Elliptic curves.

Let $p > 3$ be a prime number, and let $\mathcal{K} = \mathrm{GF}(p^n)$. We call elliptic curve, a curve $\mathcal{C} \in \mathbb{P}^2(\mathcal{K})$ defined by an equation of the form

$$x_2^2 x_0 = x_1^3 + a x_1 x_0^2 + b x_0^3.$$

In most cases we impose the condition $4a+27b \neq 0$ as a way to avoid roots with multiplicity greater then one. In most cases we will assume $x_0 = 1$. Hence, our elliptic curves will have equations of the form $y^2 = x^3 + ax + b$. In Figure 1, we show two elliptic curves when $\mathcal{K} = \mathbb{R}$.

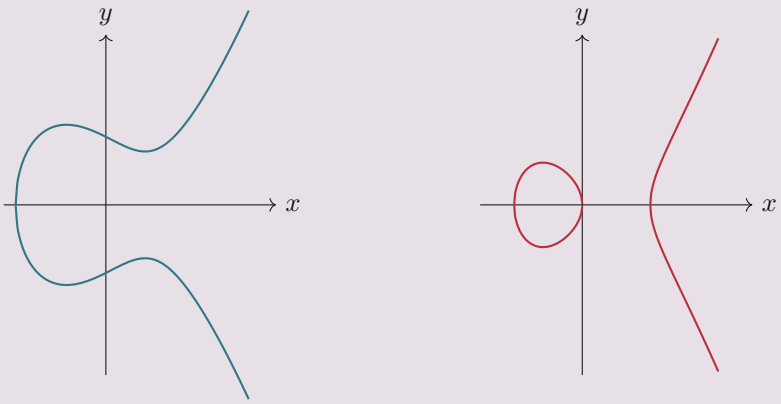


Figure 1: Example of elliptic curves over $\mathbb{R}$.

Recall we are working with discrete fields, hence the curve will have a finite amount of point. In more details, our elliptic curve $\mathcal{C}$ will represent the set of point of the fieds such that its equation is satisfied. That is,

$$\mathcal{C} = \{(x, y) \in \mathcal{K}^2 \mid y^2 = x^3 + ax + b\} \cup \Omega,$$

where Ω represents the point at infinity.

Sum of points of an elliptic curve.

Let $\mathcal{C}$ be an elliptic curve, and let $P, Q \in \mathcal{C}$. We want to know what $P + Q$ is. Differently from what one may think, the addition of points over an elliptic curve is tricky. Here, we present two techniques to apply under the proper conditions.

Rope rule: to be used when $P \neq \pm Q$, consists in computing the line l passing through P and Q, intersecting l with the equation of the curve, and once the system of equation is solved, consider the opposite of the point found[3].

Tangent rule: to be used when $P = Q$, consists in considering the tangent line to P, which we recall is

$$\frac{\partial\, F}{\partial\, x}(x - x_P) + \frac{\partial\, F}{\partial\, y}(y - y_P) = 0,$$

intersecting it with the curve and, once the system of equation is solved, consider the opposite of the point this way found.

[3]The reason we consider the opposite of the point found, has to do with the free group of divisors of the curve; a topic that is far from our interests.

Below a the formulas to be used to compute the sum of two point.

$P \neq \pm Q$**:**

$$x_{P+Q} = \left(\frac{y_P - y_Q}{x_P - x_Q}\right)^2 - x_P - x_Q$$
$$y_{P+Q} = \frac{y_P - y_Q}{x_P - x_Q}(x_P - x_{P+Q}) - y_P$$

$P = Q$**:**

$$x_{P+Q} = x_{2P} = \left(\frac{3x_P^2 + a}{2y_P}\right)^2 - 2x_p$$
$$y_{P+Q} = y_{2P} = -y_P + \frac{3x_P^2 + a}{2y_P}(x_P - x_{2P})$$

$P = -Q$**:** the point is Ω.

– 3.2 – ElGamal on elliptic curves.

Now that we know what elliptic curves are, we show how to implent the ElGamal algorithm using these special curves. The idea is the following: we encode the message to send as some point R of the curve, which is computed starting from the message itself. As it is easely understood, the strenght of this approach is that, even if the attacker gets R, retriving the original point and consequentially the message is almost impossible.

Procedure Elliptic-ElGamal shows the various steps of the algorithm.

Procedure Elliptic-ElGamal

1 Let $\mathcal{T}$ be a trusted third party. $\mathcal{T}$ chooses a prime p, and an elliptic curve $\mathcal{C}$ over GF(p), fixing a point $\gamma \in \mathcal{C}$.
2 Both $\mathcal{A}$ and $\mathcal{B}$ choose their private keys, let this be a and b respectively, and compute thei public keys, respectively $\alpha = a * \gamma$ and $\beta = b * \gamma$.
3 If m is the message $\mathcal{A}$ wants to send, $\mathcal{A}$ chooses a point $M \in \mathcal{C}$, such that the second component of M is m. Then, computes $\kappa_{AB} = a * \beta$ and transmits $c = \kappa_{AB} + M$ to B.
4 Upon receiving c, $\mathcal{B}$ first computes $\kappa_{AB} = b * \alpha$, retrieves M, and consequentially m, by computing $M = -\kappa_{AB} + c$. Recall that $-\kappa_{AB}$ is equalt to κ_{AB} with the y coordinate changed in sign.
`/* On the attacker side */`
5 $\mathcal{E}$ has no idea of neither a nor b, hence is only option is that of computing all possible multiples of γ, hoping to find either α or β.

– 4 – Digital signature.

Suppose an artist that just released some album, and asks as a way to garantee its rights on it. How should we achive such a task?

The solution is given by digital signature, which is a technique by which we can univocally connect the author of a message, a file or whatever else, to the message itselft. Thus, whether we want to garantee someones rights on a media, or for instance we have to prove that a given media was sent by a certain individual in a court room, we are able to achive such a task.

Among the several digital signature techniques, the ones we consider make use of the encryption algorithms seen so far. That is, we briefly show how to use the RSA and ElGamal algorithms to sign a document.

– 4.1 – Using RSA to sign a document.

Let m be the document $\mathcal{A}$ produced, of which we want to garantee the autenticity. Since signing the whole document is tedious, and most of the time useless, we can simply sign part of it, and to do so we consider the hash $h(m)$ of the document. Here, h is what is called an hash function.

Although the literature provides a pletora of things to say about hash functions, what matters here is simply understanding that an hash functions maps elements from a certain universe U (e.g. the set of all possible messages), to the set $\{0, \dots, m\}$, for some integer m. The important thing is that the number of collisions, i.e., the number of elements of U such that the hash coicides, is the lowest possible. Again, the literature provide us with several of such functions, thus, here we can assume h one of such hash functions.

Back to the algorithm; besides computing the hash of m, the RSA algorithm proceed as described in Section 2.1, with the difference being that instead of cyphering m, we cypher $h(w)$.

– 4.2 – Using ElGamal to sign a document.

Signing a document with ElGamal is slightly different than doing it with RSA, not only because the two algorithms are different, but also because the basic structure of the ElGamal algorithm is modified,

This different version of ElGamal, often referred to as DSA, is summarized in Procedure DSA.

Procedure DSA

1 The trusted third party $\mathcal{T}$ chooses a prime p, the ring $\mathrm{GF}(p)$ and a generator $\gamma \in \mathrm{GF}(p)$.

2 $\mathcal{A}$ is asked to choose a private key a, and computes $\alpha = \gamma^a$, which represents their public key.

3 Let m be the document. $\mathcal{A}$ is asked to compute $h(m) \in \mathbb{Z}_{p-1}$. Additionally, they choose an invertible element $k \in \mathbb{Z}_{p-1}$ and once found the inverse via the Extended Eucleadian algorithm, computes the quantities:

$$r \equiv \gamma^k \pmod{p}$$
$$s \equiv k^{-1}(h(m) + a * r) \pmod{p-1}$$

/* The pair (r, s) represents the signature */

4 To check $\mathcal{A}$'s ownership, $\mathcal{T}$ computes

$$v = \gamma^{h(m)}\alpha^r$$

If $v = r^s$ the signature is verified.

/* The issue an attacker $\mathcal{E}$ has, is due to the fact that not knowing a, it is highly likely that the pair (r, s) they found, differ from $\mathcal{A}$'s one. */

– 5 – Factorization algorithms.

It should be clear by now that most of the algorithms we've introduced rely on the complexity of several problems, the main of which is that of integer factorization. In this section we cover few algorithms that, when working[4], allow to factor an integer into it's prime factors.

– 5.1 – Pollard factorization algorithms.

Named after the British mathematician John Pollard, the two algorithms presented in this section allow, under certain hypotesis, to find one of the prime factors of a composite integer n.

The first of the two algorithms we present is *Pollard's* $p-1$ *factoring algorithm*. The algorithm is based on the notion of B-smoothness, which we define below.

Definition (B-smoothness): Let n be a composite integer, and let B a fixed bound. We say n is B-smooth, if all the prime factors of n are less or equal to B.

For instance, $n = 60$ is 5-smooth, since it's factors are $2, 3, 2, 5$.

– 5.1.1 – Pollard $p-1$ factoring algorithm.

Assume B is a smoothness bound. Let Q be the quantity

$$Q = \prod_{q \leq B} q^{\lfloor \ln n / \ln q \rfloor}$$

where each q is a prime factor smaller or equal to B. It turns out that, if p is a prime factor of n and $p-1$ is B-smooth, then $p-1|Q$. That is, for some a, $a^q \equiv 1 \pmod{p}$. That is, we find an a such that $\gcd a^Q, p = 1$. By which, if $d = \gcd a^Q - 1, n$, then $p|d$.

There is also the possibility by which $d = n$, in which case the algorithm fails. Though, the probability of this occurring lowers drastically if n is the product of at least two distinct large primes.

– 5.1.2 – Pollard ρ algorithm.

Let $f : X \to X$ be a random function, where X is a finite set with n elements. The idea behind Pollar's algorithms is very simple: since X is finite, it must happen that after a certain amount of times, $f(x) = f(x')$, for some $x, x' \in X$. The expected number of iterations preceeding a collision is $\sqrt{\pi n / 8}$.

[4]Here, when we say "if it works", we refer to the fact that the algorithm may not find a solution.

It is obvious that proceeding this way we will use a lot of space, since we will have to store each f_x. A much more efficient solution can be obtained by exploiting Floyds algorithm. The idea is again very simple, at each iteration we only store the pair (x_i, x_{2i}) until we find $x_i = x_{2i} \pmod p$. For Pollard's algorithm we set $x_0 = 2, x_{i+1} = x_i^2 + 1 \pmod p$. If it happens that $1 < \gcd x_m - x_{2m}, n < n$, then a non-trivial factor of n has been found.

References.

[1] E. R. Berlekamp. *Algebraic Coding Theory*. World Scientific, 2015.

[2] W. Diffie and M. Hellman. "New directions in cryptography". In: *IEEE Transactions on Information Theory* 22.6 (1976), pp. 644–654. DOI: 10.1109/TIT.1976.1055638.

[3] D. Hankerson, A. J. Menezes, and S. Vanstone. *Guide to Elliptic Curve Cryptography*. 1st ed. Springer, 2004.

[4] J. Hoffstein, J. Pipher, and J. H. Silverman. *An introduction to mathematical cryptography*. 2nd ed. Springer, 2014.

[5] A.J. Menezes, P.C. van Oorschot, and S.A. Vanstone. *Handbook of Applied Cryptography*. 1st ed. CRC Press, 1997.

[6] R. L. Rivest, A. Shamir, and L. Adleman. "A Method for Obtaining Digital Signatures and Public-key Cryptosystems". In: *Commun. ACM* 21.2 (Feb. 1978), pp. 120–126. DOI: 10.1145/359340.359342.